Lab 3 Assignment: Clean the NYC Taxi Data

1. Cover Page

- **Name:** Ahmed Mohamed Wageh
- **ID:** 692300198
- **Date:** 19 October 2026
- **Repository:** https://github.com/ahmedwagih2005i-bot/de-lab3-clean

---

2. The Flow

Data Pipeline Overview

The NYC Taxi data pipeline follows these stages:

1. **Source**: Raw CSV files from NYC TLC (yellow_tripdata_2021-01.csv.gz)
2. **Extract**: Read data in chunks using pandas to avoid memory issues
3. **Transform**: Clean data (remove bad dates, zero passengers, negative fares)
4. **Load**: Insert cleaned data into PostgreSQL database
5. **Serve**: Query data using pgAdmin or SQL clients

Key Components

- **ingest_data.py**: Python script that reads CSV in chunks and inserts into PostgreSQL
- **docker-compose.yaml**: Defines services (PostgreSQL, pgAdmin) and their network
- **Dockerfile**: Packages the Python script into a container image

---

3. What We Ran in Class

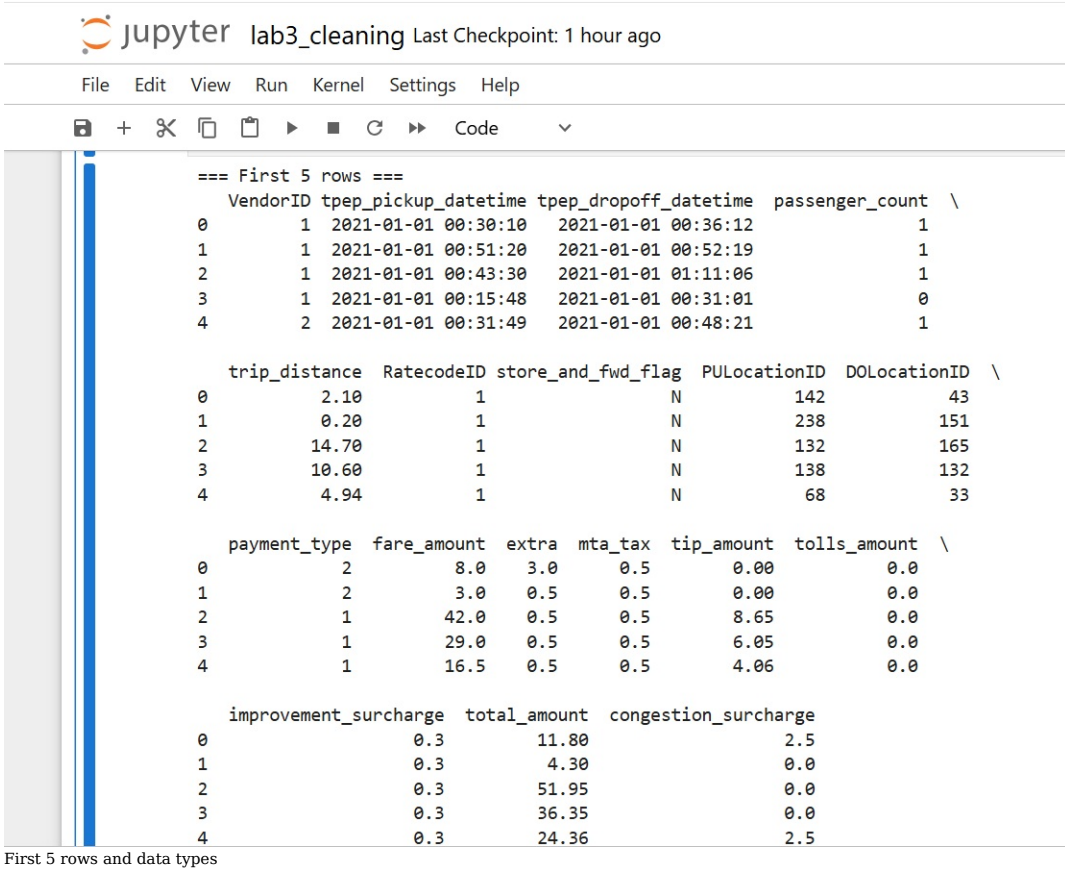
- `docker compose up`: Started PostgreSQL and pgAdmin containers
- `python ingest_data.py`: Loaded 1,369,765 rows into yellow_taxi_trips table

[Add screenshots from class here if available]

---

4. What You Found in the Data

Step 1: Look - First 100 Rows




```
=== First 5 rows ===
   VendorID tpep_pickup_datetime tpep_dropoff_datetime  passenger_count  \
0         1  2021-01-01 00:30:10   2021-01-01 00:36:12                1
1         1  2021-01-01 00:51:20   2021-01-01 00:52:19                1
2         1  2021-01-01 00:43:30   2021-01-01 01:11:06                1
3         1  2021-01-01 00:15:48   2021-01-01 00:31:01                0
4         2  2021-01-01 00:31:49   2021-01-01 00:48:21                1

   trip_distance  RatecodeID store_and_fwd_flag  PULocationID  DOLocationID  \
0           2.10           1                  N           142            43
1           0.20           1                  N           238           151
2          14.70           1                  N           132           165
3          10.60           1                  N           138           132
4           4.94           1                  N            68            33

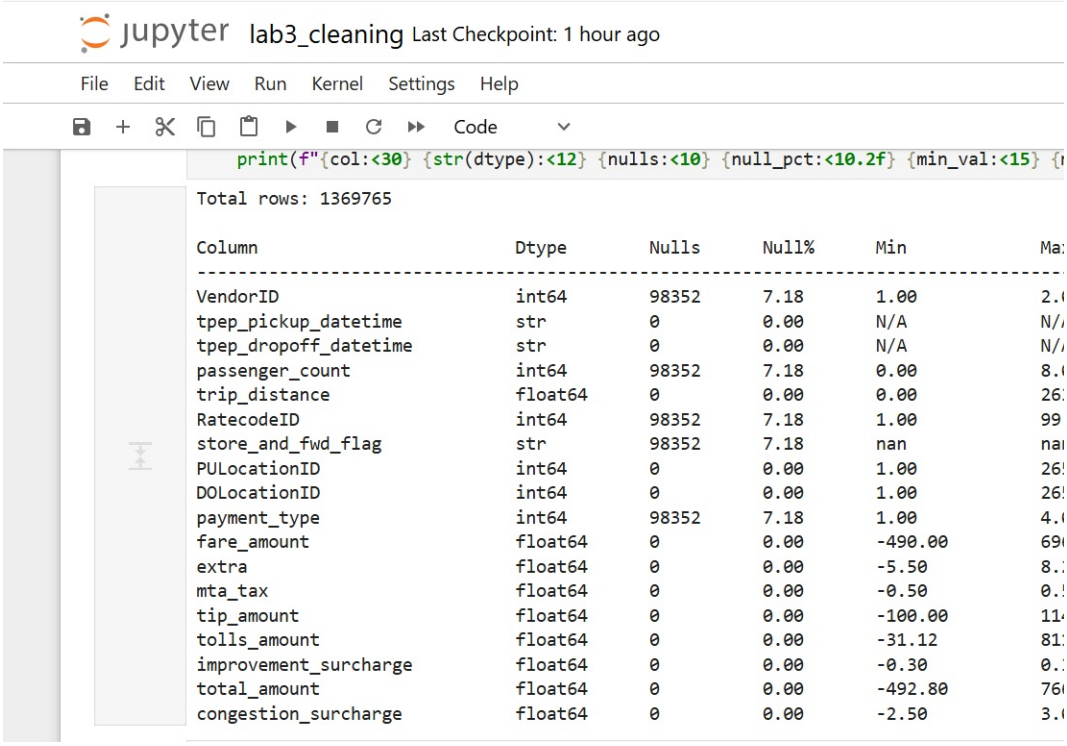
   payment_type  fare_amount  extra  mta_tax  tip_amount  tolls_amount  \
0             2          8.0    3.0      0.5        0.00           0.0
1             2          3.0    0.5      0.5        0.00           0.0
2             1         42.0    0.5      0.5        8.65           0.0
3             1         29.0    0.5      0.5        6.05           0.0
4             1         16.5    0.5      0.5        4.06           0.0

   improvement_surcharge  total_amount  congestion_surcharge
0                    0.3         11.80                   2.5
1                    0.3          4.30                   0.0
2                    0.3         51.95                   0.0
3                    0.3         36.35                   0.0
4                    0.3         24.36                   2.5
```

First 5 rows and data types

Key Observations: - Total rows in dataset: 1,369,765 - Dates are stored as strings (not datetime) - Some rows have passenger_count = 0 (row index 3) - 18 columns total

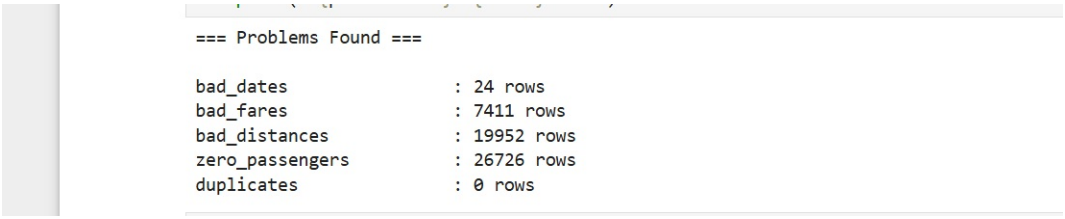
Step 2: Profile - Full File Analysis



Data profiling results

Key Findings: - **Null Values**: 5 columns have 98,352 null values each (7.18%): - VendorID, passenger_count, RatecodeID, store_and_fwd_flag, payment_type - **Data Type Issues**: - Dates stored as strings (need conversion to datetime) - fare_amount has negative values (min: -490.00) - trip_distance has extreme values (max: 263,163.28 miles)

Step 3: Problems Found



Problems discovered

Summary: - bad_dates: 24 rows (pickups outside January 2021) - bad_fares: 7,411 rows (fare_amount ≤ 0) - bad_distances: 19,952 rows (trip_distance ≤ 0) - zero_passengers: 26,726 rows (passenger_count = 0) - duplicates: 0 rows (no duplicates found)

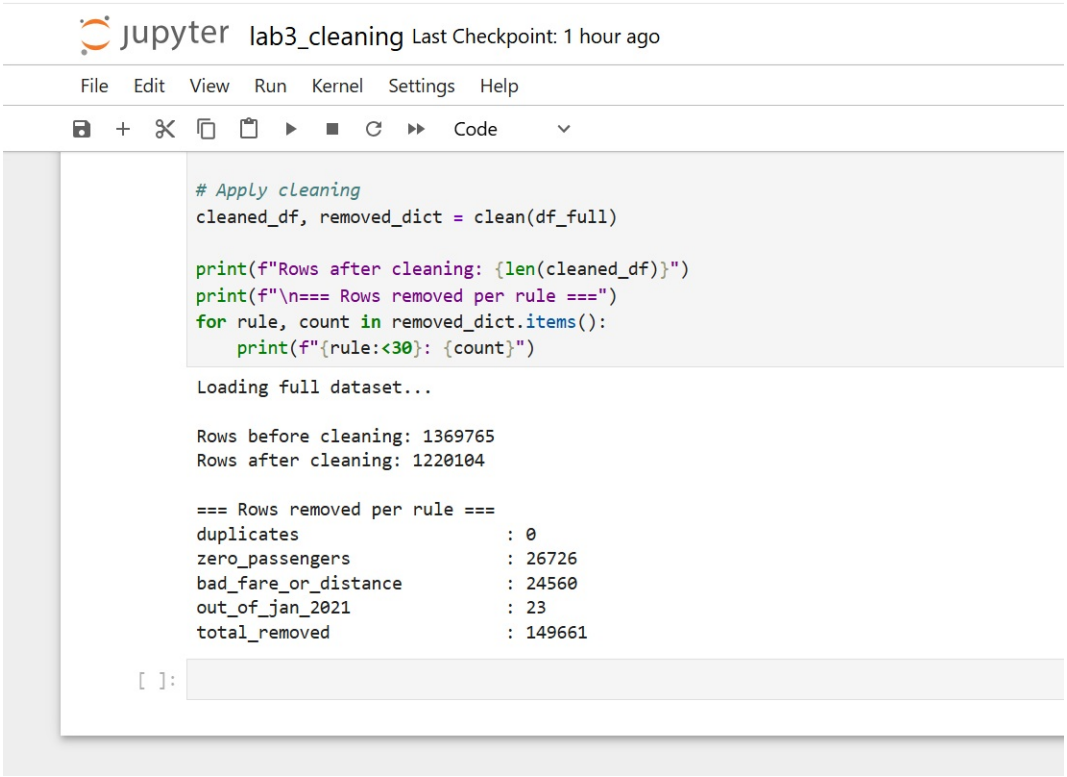
---

5. How You Cleaned It

Decisions Table

Problem	Decision	Why
duplicates	drop	No duplicates found in dataset
zero_passengers	drop	Trips with 0 passengers are illogical
bad_fare_or_distance	drop	Negative or zero fares/distances are data errors
out_of_jan_2021	drop	Trips outside January 2021 don’t belong to this dataset

Cleaning Results



Cleaning results

Statistics: - Rows before cleaning: 1,369,765 - Rows after cleaning: 1,220,104 - **Total removed: 149,661 (10.9% of data)**

Breakdown by Rule: - duplicates: 0 rows - zero_passengers: 26,726 rows - bad_fare_or_distance: 24,560 rows - out_of_jan_2021: 23 rows

---

6. Answers

Q1: Why does the container reach the DB as pgdatabase, not localhost?

Each Docker container has its own network namespace. Inside a container, `localhost` refers to the container itself, not the host machine. Docker Compose creates a network where services can reach each other by their service names. The PostgreSQL service is named `pgdatabase` in docker-compose.yaml, so other containers use `pgdatabase` as the hostname to connect to the database.

Q2: Why chunks, and what was wrong with replace for many months?

Why Chunks: The dataset is large (1.37M rows, ~24MB compressed). Loading it all at once would consume too much RAM. Reading in chunks of 100,000 rows allows processing with minimal memory usage and better performance.

Replace Problem: Using `if_exists='replace'` in pandas `to_sql()` drops the entire table and recreates it. If we process multiple months with `replace`, each month would delete the previous month's data. We need `if_exists='append'` to add data incrementally without losing existing data.

Q3: How did you verify your results?

1. **Compared row counts**: Before cleaning (1,369,765) vs after cleaning (1,220,104)
2. **Verified problematic conditions are zero**:
   - No rows with passenger_count = 0
   - No rows with fare_amount ≤ 0 or trip_distance ≤ 0
   - No rows with dates outside January 2021
3. **Checked the cleaning function**: Returns both the cleaned DataFrame and a dictionary of removed rows per rule for transparency

---

7. What Broke

Error 1: Jupyter Kernel Issue

Error Message:

ModuleNotFoundError: No module named 'pandas'

Cause: The Jupyter Notebook was using the system Python (3.10.12) instead of the virtual environment Python where pandas was installed.

Fix: Changed the kernel to `/root/de-lab2/lab3/.venv/bin/python` by clicking on the kernel selector in the top-right corner and choosing the correct Python environment.

Error 2: DtypeWarning with Mixed Types

Error Message:

Cause: The column `store_and_fwd_flag` has mixed values (sometimes 'N', sometimes True/False), causing pandas to infer different types

across chunks.

Fix: Added `low_memory=False` to `pd.read_csv()` to force pandas to read the entire file before inferring types, ensuring consistent data types across all chunks.